SmartTask : Context-Aware Mobile To-Do List for Intelligent Task Management

Group ID - 25-26J-531

Line-Up

- Introduction Video
  - Problem Introduction
  - Social & Research Impact
  - Existing Systems & Shortcomings
  - Proposed Solution: SmartTask
  - Constraints & Limitations
  - SmartTask Architecture

- Student's Information
- Knowledge Gap / Problem Definition
- Shortcomings of Existing Systems
- Creative Solution: SmartTask
- Leveraging Domains of Knowledge & Latest Technologies
- Evaluating / Validating ***SmartTask's*** Effectiveness
- Data Availability & Ethics
- System Overview: 6 - Component Architecture
- System Overview: Components Flow In Real World
- Project Requirements
- Work Breakdown Structure (WBS)
- Commercialization Potential
- References
- Q & A

Introduction Video

Problem Introduction

invideo/smarttask

The Challenge of Mobile Task M

- Endless stream of tasks & reminders
- Constant notifications break focus
- Manual reordering wastes time & effort
- Stress, missed deadlines, reduced product

Problem

Introduction Video

Problem Introduction

The Challenge of Mobile Task Management

- Endless stream of tasks & reminders
- Constant notifications break focus
- Manual reordering wastes time & effort
- Stress, missed deadlines, reduced productivity

Social & Research Impact

Why This Problem Matters

- Notification fatigue affects students & professionals
- Lowers productivity and focus
- Increases stress & missed deadlines
- Growing reliance on mobile task managers

Existing Systems & Shortcomings

Why Current Tools Fail

- Static reminders → not context-aware
- Require manual prioritization (dragging tasks up/down)
- Interrupt at wrong times (meetings, travel, etc.)
- Research studies → single-context only (e.g., location)
- Result: irrelevant reminders, more noise than value

Introduction Video

Proposed Solution: *SmartTask*

SmartTask – Smarter Tasks, Less Stress

- Context-aware task management app
- Integrates multiple signals (location, time, device state, activity)
- **Key Features:**
  - Context-based dynamic task prioritization
  - Adaptive notifications at optimal times
  - Behavioral learning → improves with use
  - Analytics dashboard for productivity insights

Constraints & Limitations

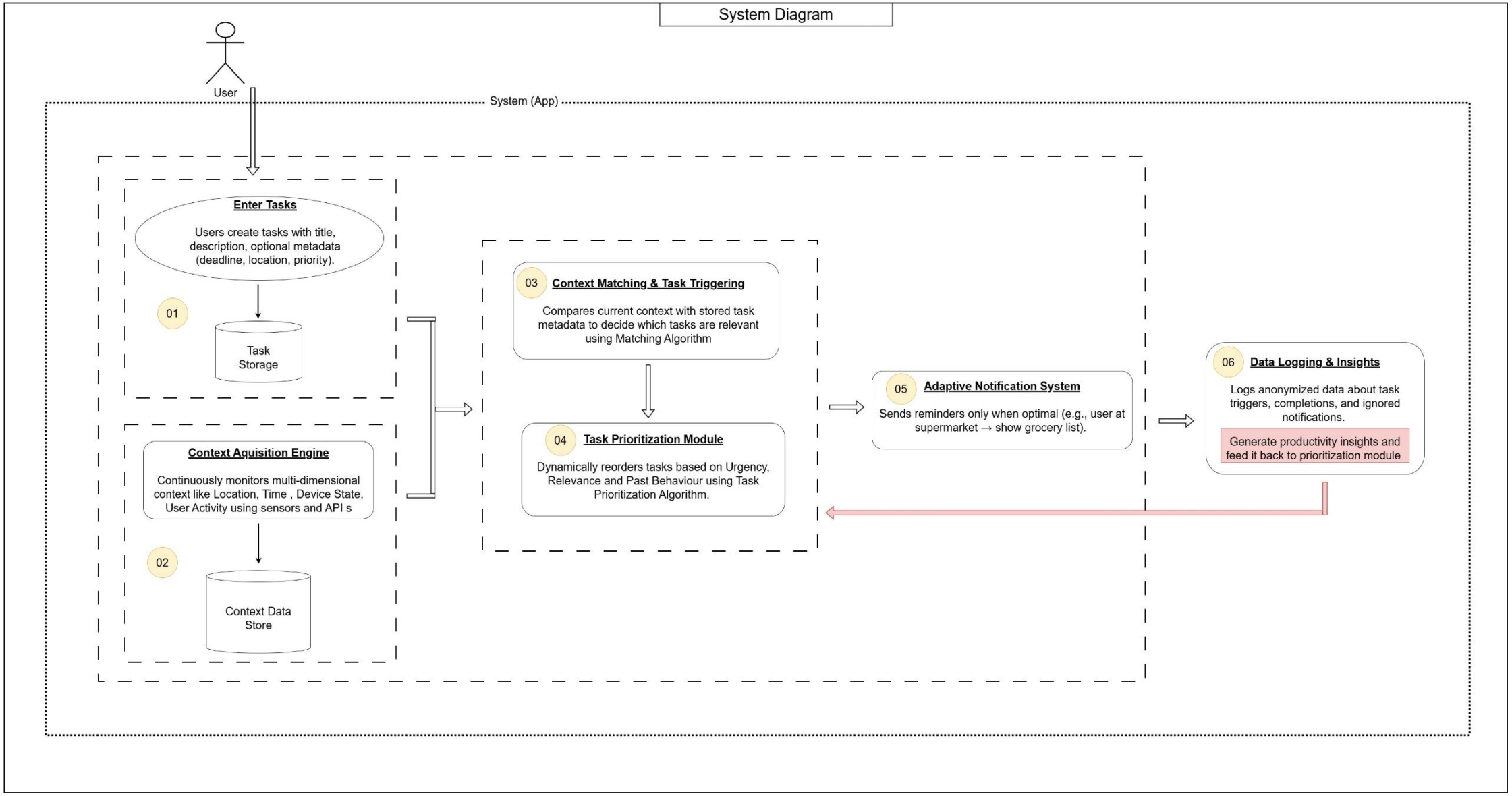
Real-World Challenges

- Battery consumption → optimized sensor polling
- Sensor reliability → fallback handling
- Privacy → local storage, user consent
- Offline operation → works with limited internet dependency

SmartTask Architecture

Modules:

1. Task Input
2. Context Acquisition
3. Context Matching & Task Triggering
4. Task Prioritization
5. Adaptive Notifications
6. Data Logging & Insights Generation (Behavioral Learning)



SmartTask : A Context-Aware Mobile Task Management System

IT21228612 | DE SILVA S.J.W.

BSc (Hons) Information Technology Specializing in Software Engineering

Knowledge Gap / Problem Definition

What's missing ?

- Traditional task managers rely on static reminders (fixed time/location). [1] [2] [3]
- They lack real-time context awareness (e.g., location, device state, user activity). [1] [2] [3]
- No adaptation to user's changing priorities, patterns, or environment. [1] [2] [3]
- Results in poor task relevance and low productivity impact. [1] [2] [3]

Shortcomings of Existing Systems

Why existing tools fail ?

- **Limited Context Awareness:** Only single-context triggers (time or location); device state and activity often ignored. [1] [2] [3] [4] [5] [6]
- **Static Task Prioritization:** Tasks must be manually reordered; urgency and behavior not considered. [1] [2] [3]
- **Notification Fatigue:** Alerts often irrelevant, causing users to ignore or disable them. [7]
- **Lack of Personalization:** Systems rarely learn from individual habits; one-size-fits-all scheduling. [4] [2]
- **Fragmented Experience:** Tasks, reminders, and calendars are not integrated intelligently; adaptive notifications missing. [1] [2] [3]

Shortcomings of Existing Systems

Feature	Castaway (2006) [4]	OASIS (2008) [5]	Task Prediction (2022) [6]	Todoist [1]	Any.do [2]	Microsoft To Do [3]	SmartTask (Proposed)
Location Awareness	✅ (map reminders)	❌	✅ (usage + location)	✅ (fixed location)	❌	❌	✅ (multi-sensor: GPS, geofencing)
Time Awareness	✅ (time triggers)	❌	✅ (temporal features)	✅ (due dates)	✅ (due dates)	✅ (due dates)	✅ (calendar + adaptive scheduling)
User Activity Awareness	❌	✅ (cognitive breakpoints)	✅ (app usage patterns)	❌	❌	❌	✅ (motion sensors + cognitive context)
Device State Awareness (battery, connectivity)	❌	❌	❌	❌	❌	❌	✅ (battery, connectivity-aware task scheduling)
Dynamic Prioritization	❌	❌	❌ (predictive ranking)	❌	❌	✅ (My Day only)	✅ (real-time, multi-factor prioritization)
Adaptive Notifications	✅ (time/place only)	✅ (breakpoint-based)	❌ (predicted optimal times)	❌	❌	❌	✅ (context-aware + interruption management)
Learning from Behavior	❌	✅ (learns breakpoints)	✅ (learns task patterns)	✅ (basic location suggestions)	❌	✅ (task suggestions)	✅ (continuous feedback loop with behavioral learning)

Creative Solution: ***SmartTask***

How ***SmartTask*** bridges the gap

- **Context Acquisition:** Real-time signals (location, time, device state, activity).
- **Dynamic Prioritization:** Tasks ranked by urgency + relevance.
- **Adaptive Notifications:** Alerts delivered at the most effective moments.
- **Behavioral Learning:** System improves continuously via feedback (completed/dismissed tasks).

Leveraging Domains of Knowledge & Latest Technologies

Key Domains of Knowledge

- **Mobile App Development:** Native Android app for full access to sensors, APIs, and high performance.
- **Context-Aware Computing:** Real-time sensing of location, time, device state, and user activity for adaptive task management.
- **Algorithm Development & Behavioral Learning:** Dynamic task prioritization and learning from user interactions.
- **Human-Computer Interaction (HCI):** Intuitive UI for minimal cognitive load, easy interaction, and better adoption.

Leveraging Domains of Knowledge & Latest Technologies

Main Domain	Selected Technologies	Justification
Mobile App Development	• Java (Android native) • Android Studio • Android Emulator + Physical Devices • JUnit, Espresso, Mockito • Git + GitHub • Android smartphones + PC	• Native app for full sensor/API access • Official IDE with debugging and build tools • Testing across devices and Android versions • Unit/UI testing for reliability • Version control and CI/CD • Real devices for validating notifications and sensors
Context-Aware Computing	• Google Maps API • Android Location & Activity APIs • Calendar API • SQLite / Room DB • Android Notification API	• Accurate Location, activity, and calendar-based context • Local storage for tasks and context • Adaptive notifications to reduce fatigue
Algorithm & Behavioral Learning	• Custom Java logic • Lightweight ML • Custom scoring algorithms • SQLite queries • Java Collections/Streams • Optional Weka / DL4J	• Context-based task matching • Dynamic prioritization and learning from behavior • Lightweight on-device analysis, optional advanced ML
Human-Computer Interaction (HCI)	• Figma	• Intuitive UI/UX for minimal cognitive load and easy interaction • Free Tool • Provides prototyping features • Large community ; More Support

Evaluating / Validating *SmartTask*'s Effectiveness

Evaluation Metrics / Methods

- **Task Completion Rate:** Measure the percentage of tasks completed on time vs. missed.
- **Reduction in Missed Reminders:** Compare traditional static reminders vs. SmartTask's adaptive notifications.
- **User Cognitive Load:** Assess using NASA-TLX survey to see if ***SmartTask*** reduces mental effort. (NASA Task Load Index)
- **User Feedback / Satisfaction:** Short survey on usefulness, intuitiveness, and perceived productivity.
- **Simulation / Real-World Testing:**
  - Use Android Emulator and real devices.
  - Test tasks triggered by various contexts (location, activity, calendar events).

Data Availability & Ethics

Ethical Data Handling in *SmartTask*

Data Sources:

- Device sensors: GPS, accelerometer, gyroscope & API s.
- Calendar & scheduled events.
- Task metadata (title, description, priority, deadlines).

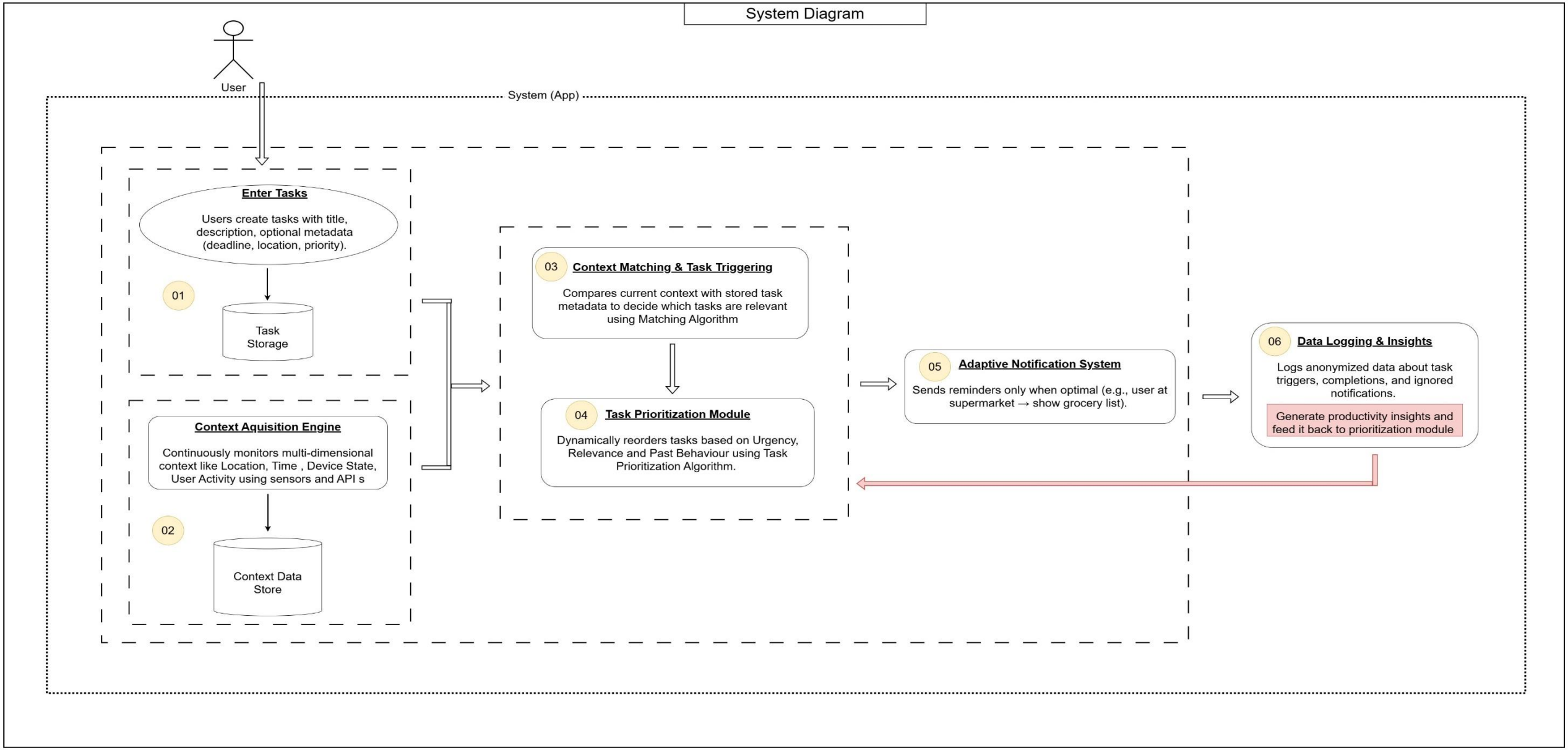
Ethical Measures:

- **Local Storage Only:** Context and task data stored on-device (SQLite/Room).
- **Anonymized Data:** No personal identifiers sent externally. Data will be anonymized before storing.
- **User Control:** Users can delete all stored tasks and context data anytime.
- **Minimal Battery Impact:** Context sampling optimized to avoid excessive power consumption.
- **User Consent:** Inform users about data collection through a privacy policy agreement during app installation.

Justification:

- Ensures privacy and security, complying with ethical standards for personal mobile data.
- Balances rich context-aware features with user trust and consent.

System Overview: 6 - Component Architecture



System Overview: Components Flow In Real World

Scenario: A student has 3 upcoming tasks today:

1. Finish Assignment – Deadline: 2:00 PM | Location: Library | Priority: High
2. Buy Stationery – Deadline: Evening | Location: Bookshop | Priority: Medium
3. Call Family – Deadline: Night | Location: Home | Priority: Low

1 - Task Input Module

- Student enters tasks with metadata: title, deadline, location, priority.
- Example: "Finish Assignment – 2:00 PM – Library – High Priority".
- Stored in Task Database.

2 - Context Acquisition Engine

- Continuously gathers real-time signals:
  - Location → detects when student is at home or library.
  - Time → checks system clock (e.g., 1:30 PM, 30 mins before deadline).
  - Device state → battery at 20% triggers early reminder.
  - Activity → detects "walking" → student is moving toward library.
- Stored in Context Data Store.

3 - Context Matching & Task Triggering

- Matches current context with task metadata.
- Example: At 1:30 PM, location = near library, time = 30 mins to deadline.
- Filters task list → highlights "Finish Assignment" as relevant.

4 - Task Prioritization Module

- Orders tasks dynamically:
  - Finish Assignment → top priority (deadline + location match).
  - Buy Stationery → lower (not urgent now).
  - Call Family → lowest (later at night).
- Works in loop with insights to refine ranking.

5 - Adaptive Notification System

- Sends just-in-time reminder:
  - "You're near the library, time to finish your assignment before 2:00 PM."
- Suppresses irrelevant notifications (Call Family won't pop up now).

6 - Data Logging & Insights

- Logs: student opened the reminder, completed assignment at 1:55 PM.
- Stores response + context.
- Feeds back into prioritization → system learns optimal reminder timing (e.g., 30 mins before worked well).

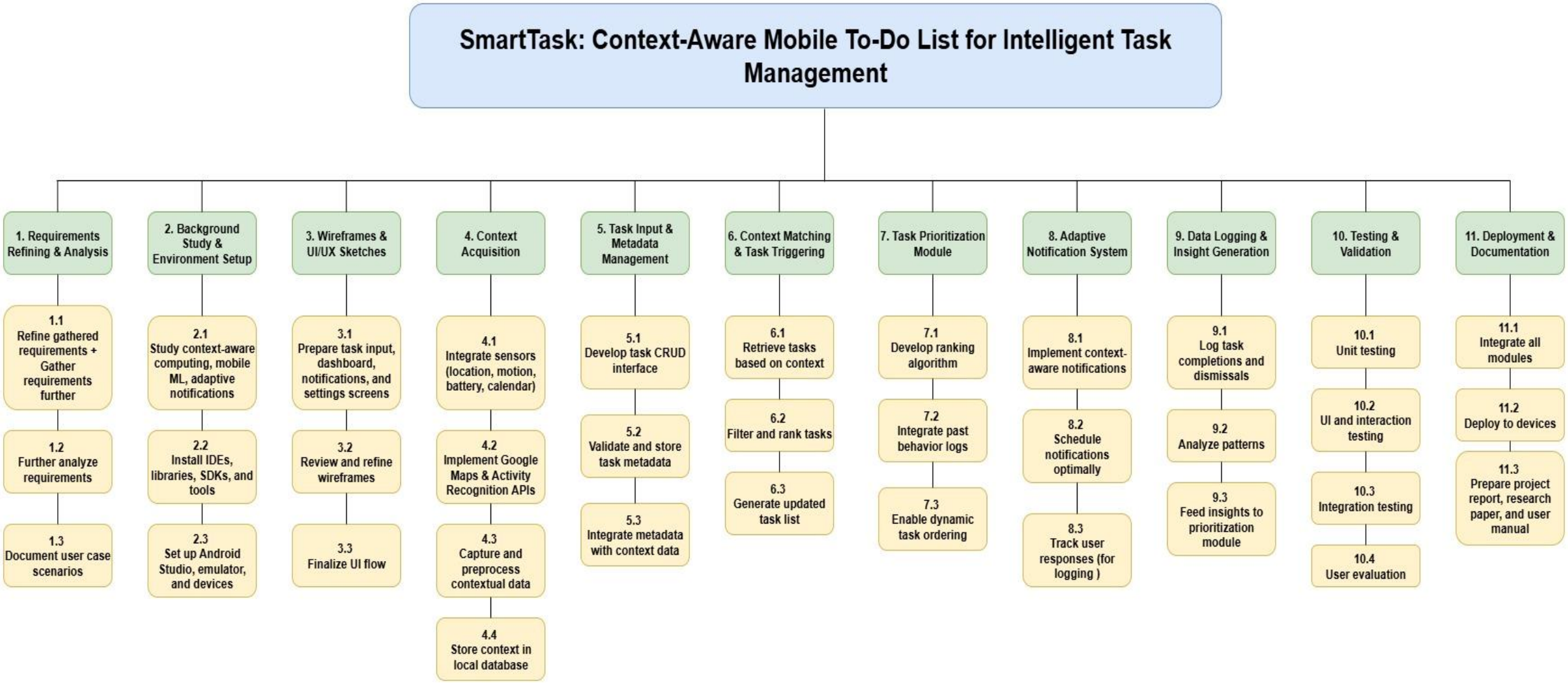
Result:

- Student finishes assignment on time with minimal distractions.
- Notifications felt timely and useful.
- System adapts to user behavior for future days.

Project Requirements

Requirement Type	Key Requirements
Functional	- Create, edit, delete tasks with metadata (title, deadline, location, priority) - Context acquisition (location, time, device state, user activity) - Real-time context matching & task triggering- Dynamic prioritization (auto + manual) - Adaptive notifications (smart timing, suppression) - Behavioral learning & analytics (completion trends) - Intuitive UI for tasks, lists, and summaries
Non-Functional	- Real-time response (<1s latency) - Minimal, user-friendly interface - Scalable & modular architecture - Reliable with offline support (SQLite/Room) - Data privacy (consent, encryption) - Maintainable code (modular, documented) - Battery & resource efficient
User	- Easy task creation & management - Relevant, non-intrusive notifications - View history & productivity insights - Works offline on Android devices - Control over notifications (quiet hours) - System adapts to personal behavior
System	- Android smartphone (Android 10+, GPS, sensors, 100MB storage) - Development via Android Studio + Java SDK - Local DB (SQLite/Room), Android APIs - Optional ML libs for behavior modeling - GitHub for version control, JUnit/Espresso for testing

Work Breakdown Structure (WBS)



Commercialization Potential

Business Model – How We Make Money

- **Differentiation:** Unlike other apps, ***SmartTask*** is adaptive & context-aware, offering intelligent task prioritization and notifications tuned to real-life conditions.
- **Investment & Cost:**
  - Labor: Development hours for AI algorithms, UI/UX, backend.
  - Material: Android devices for testing.
  - API s : Subscriptions Cost for API s (e.g. google maps).
  - Recovery: Freemium approach – free version with ads; premium subscription offers advanced insights and an ad-free experience.
- **Target Market:** Students, busy professionals, productivity enthusiasts.
- **Customer Profile:** Mobile-first users with heavy task-list dependency and fast-paced lifestyles.
- **Value Proposition:** Reduce stress, improve productivity, and deliver notifications at the right moment, not just the next moment.
- **IP Considerations:** Proprietary algorithms for context-matching and adaptive notification timing create defensible intellectual property.

References

[1] Todoist – A To-Do List to Organize Your Work & Life (2025). [Online]. Available: https://www.todoist.com/.

[2] Any.do, “How to Manage Notifications in Any.do,” (2023). [Online]. Available: https://support.any.do/en/articles/8610107-how-to-manage-notifications-in-any-do.

[3] Microsoft, Microsoft To Do: Task Management App (2025). [Online]. Available: https://www.microsoft.com/en-us/microsoft-365/microsoft-to-do.

[4] A. Kessell and C. Chan, “Castaway: A Context-Aware Task Management System,” in CHI 2006 Ext. Abstracts, 2006, pp. 741–746.

[5] S. T. Iqbal, A Framework for Intelligent Notification Management in Multitasking Domains, Ph.D. dissertation, Univ. Illinois at Urbana-Champaign, 2008.

[6] Y. Tian, Z. Wang, D. Pelleg et al., “Characterization and prediction of mobile tasks,” ACM Trans. Inf. Syst., vol. 41, no. 1, 2022.

[7] H. Winghart and A. Al Zaidy, “MageMemo: A gamified productivity tool for user engagement,” J. Information Technology, Cybersecurity, and AI, vol. 1, no. 1, pp. 9–17, 2024.

Q & A ??

Thank You !!!